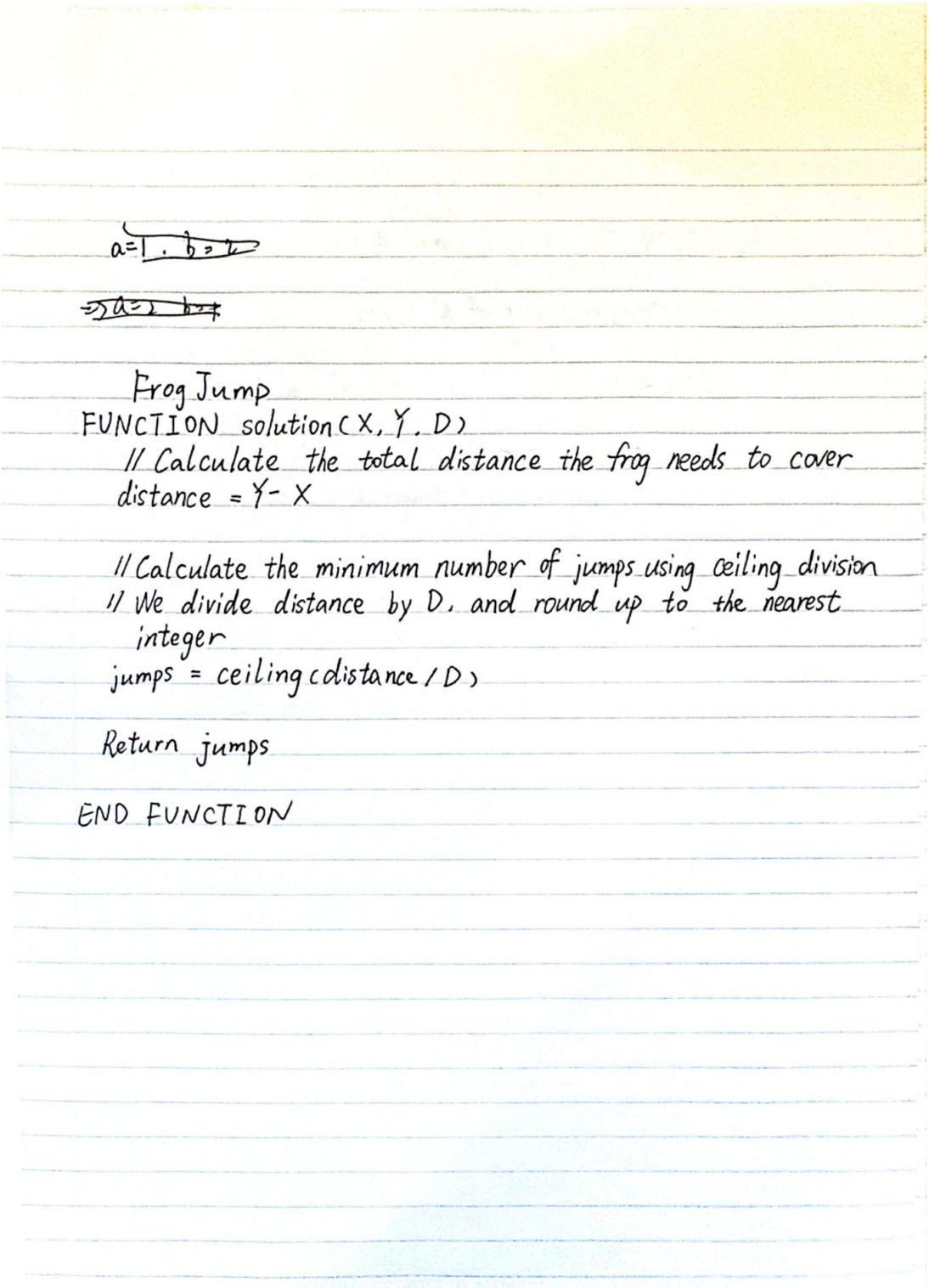
Codility Practice: Five Problems

Name: ColumbinaSnowy
GitHub repository: https://github.com/ColumbinaSnowy/Assignment1
AI conversation: https://claude.ai/code/session_014yRSkwwWRBS2RYejMHn1up

1. FrogJmp

Problem: https://app.codility.com/programmers/lessons/3-time_complexity/frog_jmp/

1. Handwritten pseudocode



2. Python code

```
# FrogJmp (Codility Lesson 3 - Time Complexity)
# Time: O(1), Space: O(1)


def solution(X, Y, D):
    # Calculate the total distance the frog needs to cover
    distance = Y - X

    # Calculate the minimum number of jumps using ceiling division
    # (integer arithmetic avoids floating-point rounding errors)
    jumps = (distance + D - 1) // D

    return jumps
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/3-time_complexity/frog_jmp/
AI tool: Claude Code (Anthropic), used to help draft the solution and pseudocode.

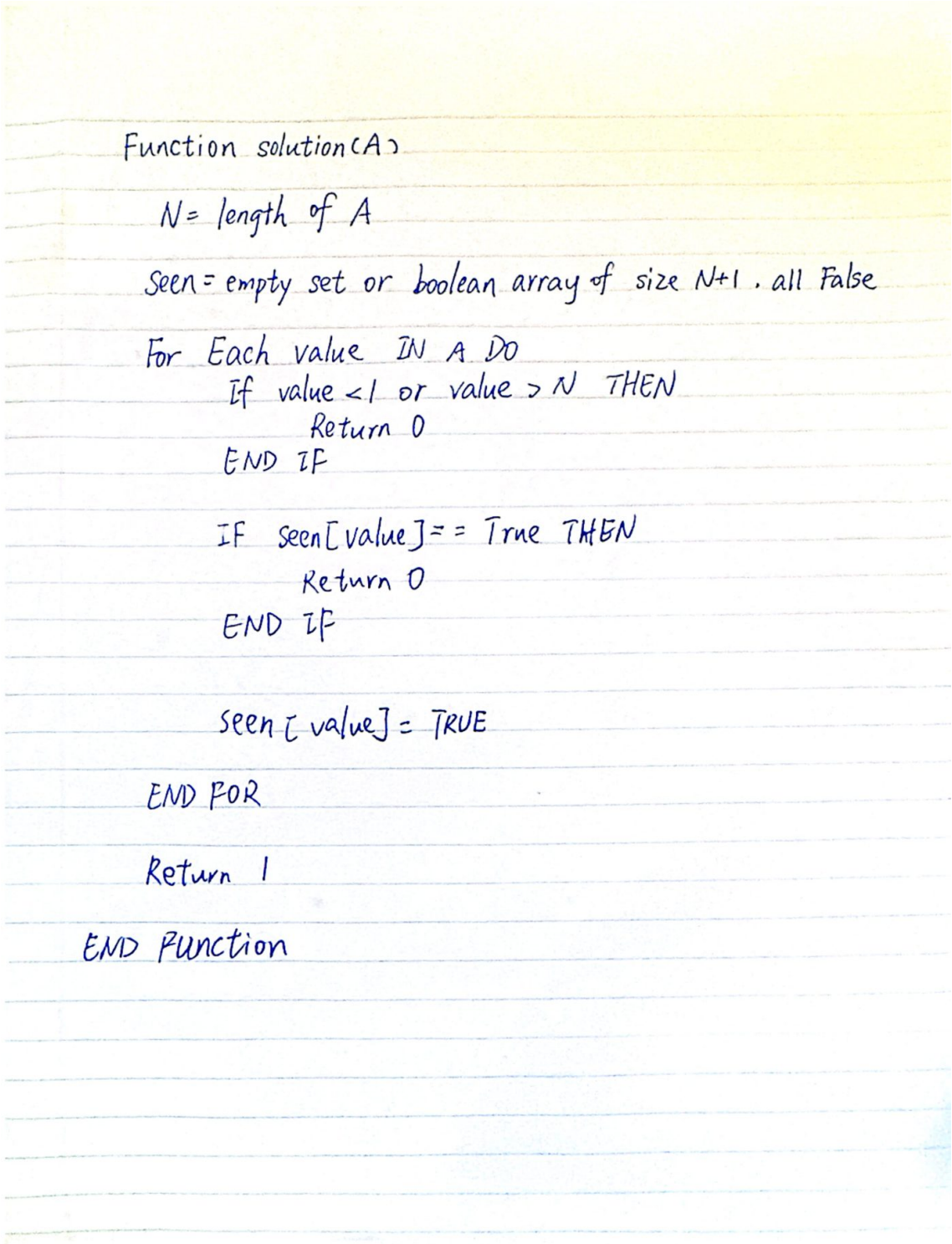
4. Links

GitHub: https://github.com/ColumbinaSnowy/Assignment1
AI conversation: https://claude.ai/code/session_014yRSkwwWRBS2RYejMHn1up

2. PermCheck

Problem: https://app.codility.com/programmers/lessons/4-counting_elements/perm_check/

1. Handwritten pseudocode



```
Function solution(A)
    N = length of A
    Seen = empty set or boolean array of size N+1 . all False
    For Each value IN A DO
        If value < 1 or value > N THEN
            Return 0
        END IF

        IF Seen[value] == True THEN
            Return 0
        END IF

        Seen[value] = TRUE
    END FOR
    Return 1
END Function
```

2. Python code

```
# PermCheck (Codility Lesson 4 - Counting Elements)
# Time: O(N), Space: O(N)


def solution(A):
    N = len(A)
    seen = [False] * (N + 1)

    for value in A:
        # Value outside 1..N -> cannot be a permutation
        if value < 1 or value > N:
            return 0
        # Duplicate value -> cannot be a permutation
        if seen[value]:
            return 0
        seen[value] = True

    return 1
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/4-counting_elements/perm_check/
AI tool: Claude Code (Anthropic), used to help draft the solution and pseudocode.

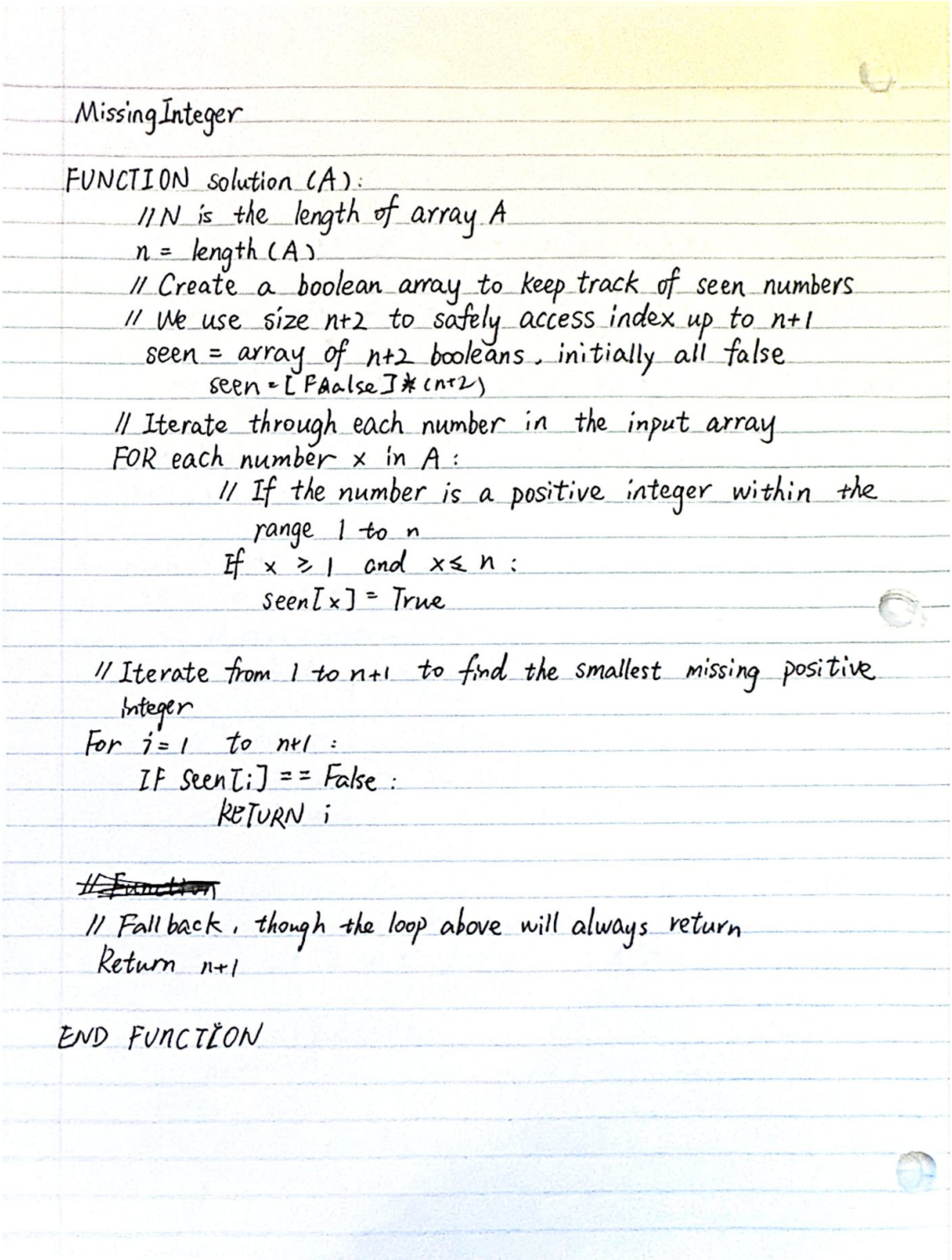
4. Links

GitHub: https://github.com/ColumbinaSnowy/Assignment1
AI conversation: https://claude.ai/code/session_014yRSkwwWRBS2RYejMHn1up

3. MissingInteger

Problem: https://app.codility.com/programmers/lessons/4-counting_elements/missing_integer/

1. Handwritten pseudocode



MissingInteger

```
FUNCTION solution (A):
    // N is the length of array A
    n = length (A)
    // Create a boolean array to keep track of seen numbers
    // We use size n+2 to safely access index up to n+1
     seen = array of n+2 booleans, initially all false
         seen = [False]*(n+2)
    // Iterate through each number in the input array
    FOR each number x in A:
            // If the number is a positive integer within the
               range 1 to n
             If x ≥ 1 and x ≤ n:
                seen[x] = True

    // Iterate from 1 to n+1 to find the smallest missing positive
       integer
    For i = 1 to n+1 :
        IF seen[i] == False :
            RETURN i

    // Function
    // Fallback, though the loop above will always return
     Return n+1

END FUNCTION
```

2. Python code

```
# MissingInteger (Codility Lesson 4 - Counting Elements)
# Time: O(N), Space: O(N)


def solution(A):
    # n is the length of array A
    n = len(A)

    # Boolean array to keep track of seen numbers
    # Size n + 2 so we can safely access index up to n + 1
    seen = [False] * (n + 2)

    # Mark every positive integer within the range 1..n
    for x in A:
        if 1 <= x <= n:
            seen[x] = True

    # Iterate from 1 to n + 1 to find the smallest missing positive integer
    for i in range(1, n + 2):
        if not seen[i]:
            return i

    # Fallback, though the loop above will always return
    return n + 1
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/4-counting_elements/missing_integer/
AI tool: Claude Code (Anthropic), used to help draft the solution and pseudocode.

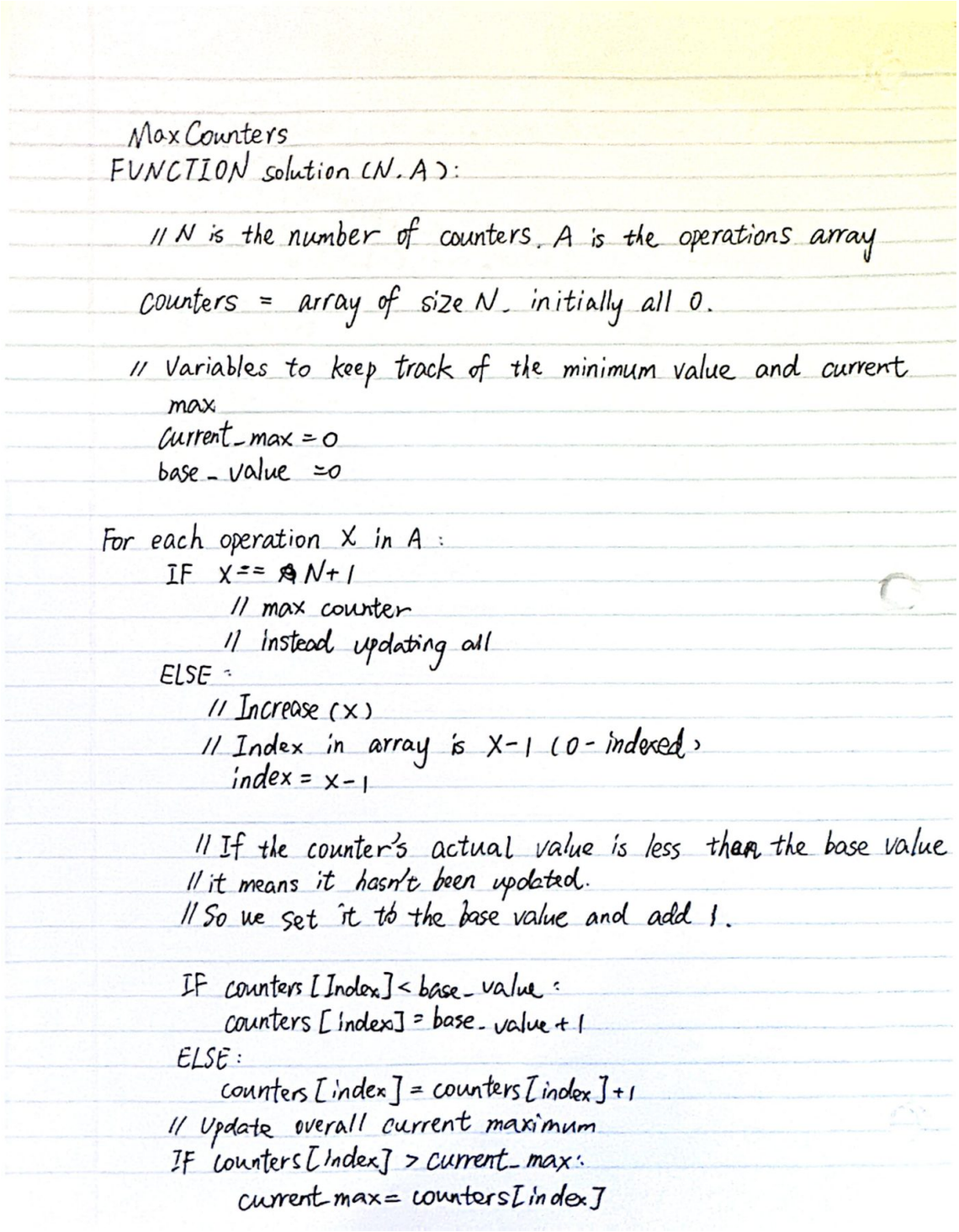
4. Links

GitHub: https://github.com/ColumbinaSnowy/Assignment1
AI conversation: https://claude.ai/code/session_014yRSkwwWRBS2RYejMHn1up

4. MaxCounters

Problem: https://app.codility.com/programmers/lessons/4-counting_elements/max_counters/

1. Handwritten pseudocode



```
MaxCounters
FUNCTION solution(N, A):

  // N is the number of counters. A is the operations array

  counters = array of size N, initially all 0.

  // Variables to keep track of the minimum value and current
     max
  current_max = 0
  base_value = 0

For each operation X in A:
    IF X == N+1
        // max counter
        // instead updating all
    ELSE:
      // Increase(X)
      // Index in array is X-1 (0-indexed)
        index = X-1

       // If the counter's actual value is less than the base value
      // it means it hasn't been updated.
      // So we set it to the base value and add 1.

      IF counters[Index] < base_value:
          counters[Index] = base_value + 1
      ELSE:
          counters[index] = counters[index] + 1
      // Update overall current maximum
      IF counters[Index] > current_max:
          current_max = counters[index]
```

```
// Final pass to apply the base_value to any counters that were
  left behind
  For i from 0 to N-1:
        If counters[i] < base_value:
            counters[i] = base_value

  Return counters

END FUNCTION
```

2. Python code

```
# MaxCounters (Codility Lesson 4 - Counting Elements)
# Time: O(N + M), Space: O(N)


def solution(N, A):
    # N is the number of counters, A is the operations array
    counters = [0] * N

    # Variables to keep track of the minimum value and current max
    current_max = 0
    base_value = 0

    for X in A:
        if X == N + 1:
            # max counter: instead of updating all counters,
            # just record the new base value (lazy update)
            base_value = current_max
        else:
            # increase(X); index in array is X - 1 (0-indexed)
            index = X - 1

            # If the counter's actual value is less than the base value,
            # it hasn't been updated since the last max counter,
            # so set it to the base value and add 1.
            if counters[index] < base_value:
                counters[index] = base_value + 1
            else:
                counters[index] = counters[index] + 1

            # Update overall current maximum
            if counters[index] > current_max:
                current_max = counters[index]

    # Final pass to apply the base value to any counters left behind
    for i in range(N):
        if counters[i] < base_value:
            counters[i] = base_value

    return counters
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/4-counting_elements/max_counters/
AI tool: Claude Code (Anthropic), used to help draft the solution and pseudocode.

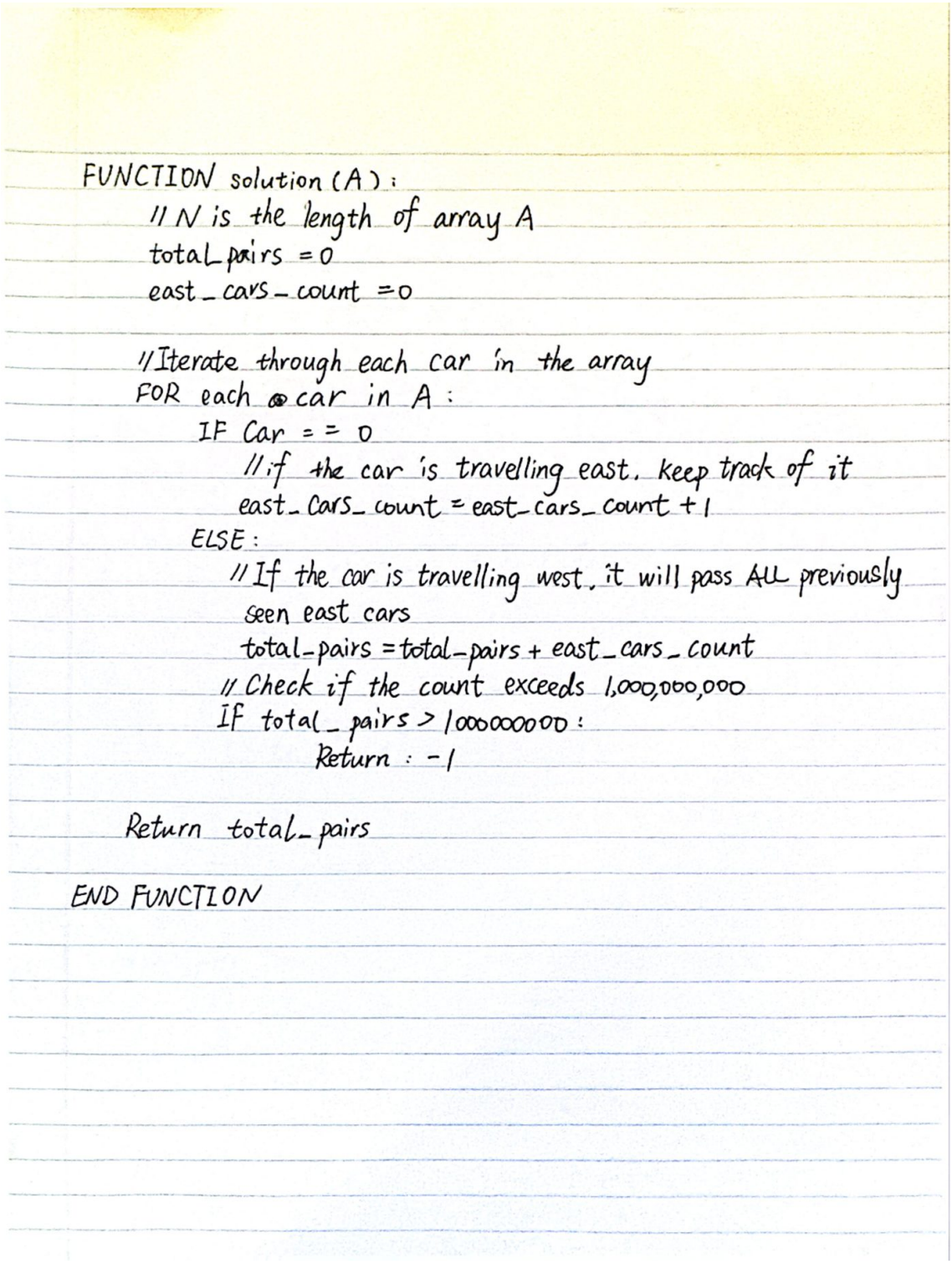
4. Links

GitHub: https://github.com/ColumbinaSnowy/Assignment1
AI conversation: https://claude.ai/code/session_014yRSkwwWRBS2RYejMHn1up

5. PassingCars

Problem: https://app.codility.com/programmers/lessons/5-prefix_sums/passing_cars/

1. Handwritten pseudocode



```
FUNCTION solution (A):
    // N is the length of array A
    total_pairs = 0
    east_cars_count = 0

    //Iterate through each car in the array
    FOR each car in A:
        IF Car == 0
            //if the car is travelling east, keep track of it
            east_cars_count = east_cars_count + 1
        ELSE:
            // If the car is travelling west, it will pass ALL previously
            seen east cars
            total_pairs = total_pairs + east_cars_count
            // Check if the count exceeds 1,000,000,000
            If total_pairs > 1000000000:
                Return : -1

    Return total_pairs

END FUNCTION
```

2. Python code

```
# PassingCars (Codility Lesson 5 - Prefix Sums)
# Time: O(N), Space: O(1)


def solution(A):
    total_pairs = 0
    east_cars_count = 0

    for car in A:
        if car == 0:
            # Car travelling east: keep track of it
            east_cars_count += 1
        else:
            # Car travelling west passes ALL previously seen east cars
            total_pairs += east_cars_count
            # Check if the count exceeds 1,000,000,000
            if total_pairs > 1000000000:
                return -1

    return total_pairs
```

3. Citations

Problem statement: Codility Lessons,
https://app.codility.com/programmers/lessons/5-prefix_sums/passing_cars/
AI tool: Claude Code (Anthropic), used to help draft the solution and pseudocode.

4. Links

GitHub: https://github.com/ColumbinaSnowy/Assignment1
AI conversation: https://claude.ai/code/session_014yRSkwwWRBS2RYejMHn1up